

EC4219: Software Engineering

Lecture 2 — Propositional Logic (2) *Normal Forms and DPLL*

Sunbeom So
2025 Spring

Overview

- Goal: An algorithm called **DPLL** for determining satisfiability.
 - ▶ Many SAT solvers used today are based on DPLL.
 - ▶ SAT solver: software that solves the boolean satisfiability problem (theorem prover for propositional logic)
- DPLL requires converting formulas to a representation called **normal forms**.
- Thus, we will study normal forms first and then DPLL.

Normal Forms

- A normal form of formulas is a certain syntactic restriction such that, for every formula F of the logic, there is an equivalent formula F' .
- Three important normal forms for propositional logic.
 - ▶ Negation Normal Form (NNF)
 - ▶ Disjunctive Normal Form (DNF)
 - ▶ Conjunctive Normal Form (CNF)

Negation Normal Form (NNF)

- NNF requires that (1) $\neg$, $\wedge$, and $\vee$ are the only connectives (i.e., no $\rightarrow$ and $\leftrightarrow$) and (2) negations appear only in front of atoms (negations appear only in literals).
 - ▶ Is $P \wedge Q \wedge (R \vee \neg S)$ in NNF?
 - ▶ Is $\neg P \vee \neg(P \wedge Q)$ in NNF?
 - ▶ Is $\neg\neg P \wedge Q$ in NNF?
- Transforming a formula F to an equivalent formula F' in NNF can be done by repeatedly applying the following list of template equivalences:

$$\begin{array}{lll} \neg\neg F_1 & \iff & F_1 \\ \neg \top & \iff & \perp \\ \neg \perp & \iff & \top \\ \neg(F_1 \wedge F_2) & \iff & \neg F_1 \vee \neg F_2 \\ \neg(F_1 \vee F_2) & \iff & \neg F_1 \wedge \neg F_2 \\ F_1 \rightarrow F_2 & \iff & \neg F_1 \vee F_2 \\ F_1 \leftrightarrow F_2 & \iff & (F_1 \rightarrow F_2) \wedge (F_2 \rightarrow F_1) \end{array}$$

Negation Normal Form (NNF)

- NNF requires that (1) $\neg$, $\wedge$, and $\vee$ are the only connectives (i.e., no $\rightarrow$ and $\leftrightarrow$) and (2) negations appear only in literals (i.e., negations appear only in front of atoms).
 - ▶ Is $P \wedge Q \wedge (R \vee \neg S)$ in NNF? **O**
 - ▶ Is $\neg P \vee \neg(P \wedge Q)$ in NNF? **X**
 - ▶ Is $\neg\neg P \wedge Q$ in NNF? **X**
- Transforming a formula F to an equivalent formula F' in NNF can be done by repeatedly applying the list of template equivalences below:

$$\begin{array}{lll} \neg\neg F_1 & \iff & F_1 \\ \neg\top & \iff & \perp \\ \neg\perp & \iff & \top \\ \neg(F_1 \wedge F_2) & \iff & \neg F_1 \vee \neg F_2 \\ \neg(F_1 \vee F_2) & \iff & \neg F_1 \wedge \neg F_2 \\ F_1 \rightarrow F_2 & \iff & \neg F_1 \vee F_2 \\ F_1 \leftrightarrow F_2 & \iff & (F_1 \rightarrow F_2) \wedge (F_2 \rightarrow F_1) \end{array}$$

Example: NNF

Convert $F : \neg(P \rightarrow \neg(P \wedge Q))$ into NNF.

- By the template equivalence $F_1 \rightarrow F_2 \iff \neg F_1 \vee F_2$, we produce

$$F' : \neg(\neg P \vee \neg(P \wedge Q))$$

- Applying the template $\neg(F_1 \vee F_2) \iff \neg F_1 \wedge \neg F_2$, we produce

$$F'' : \neg\neg P \wedge \neg\neg(P \wedge Q)$$

- Finally, applying the template equivalence $\neg\neg F_1 \iff F_1$ to each conjunct produces

$$F''' : P \wedge P \wedge Q$$

F''' is in NNF and is equivalent to F .

Disjunctive Normal Form (DNF)

- A formula is in disjunctive normal form (DNF) if it is a disjunction of conjunctive clauses (conjunctions of literals):

$$\bigvee_i \bigwedge_j l_{i,j}$$

- To convert a formula F into an equivalent formula in DNF,
 - (1) transform F into NNF, and then
 - (2) distribute conjunctions over disjunctions:

$$\begin{aligned}(F_1 \vee F_2) \wedge F_3 &\iff (F_1 \wedge F_3) \vee (F_2 \wedge F_3) \\ F_1 \wedge (F_2 \vee F_3) &\iff (F_1 \wedge F_2) \vee (F_1 \wedge F_3)\end{aligned}$$

Example: DNF

To convert

$$F : (Q_1 \vee \neg\neg Q_2) \wedge (\neg R_1 \rightarrow R_2)$$

into DNF,

- Transform F into NNF.

$$F' : (Q_1 \vee Q_2) \wedge (R_1 \vee R_2)$$

- Apply distributivity.

$$F'' : (Q_1 \wedge (R_1 \vee R_2)) \vee (Q_2 \wedge (R_1 \vee R_2))$$

Apply distributivity again.

$$F''' : (Q_1 \wedge R_1) \vee (Q_1 \wedge R_2) \vee (Q_2 \wedge R_1) \vee (Q_2 \wedge R_2)$$

F''' is in DNF and is equivalent to F .

Conjunctive Normal Form (CNF)

- A formula is in conjunctive normal form (CNF) if it is a conjunction of disjunctive clauses (disjunctions of literals):

$$\bigwedge_i \bigvee_j l_{i,j}$$

- To convert a formula F into an equivalent formula in CNF,
 - (1) transform F into NNF, and then
 - (2) distribute disjunctions over conjunctions:

$$\begin{aligned}(F_1 \wedge F_2) \vee F_3 &\iff (F_1 \vee F_3) \wedge (F_2 \vee F_3) \\ F_1 \vee (F_2 \wedge F_3) &\iff (F_1 \vee F_2) \wedge (F_1 \vee F_3)\end{aligned}$$

Example: CNF

Convert $F : (Q_1 \wedge \neg\neg Q_2) \vee (\neg R_1 \rightarrow R_2)$ into CNF.

- Transform F into NNF.

$$F' : (Q_1 \wedge Q_2) \vee (R_1 \vee R_2)$$

- Apply distributivity.

$$F'' : (Q_1 \vee R_1 \vee R_2) \wedge (Q_2 \vee R_1 \vee R_2)$$

F'' is in CNF and is equivalent to F .

Decision Procedures

- A **decision procedure** decides whether F is satisfiable or not after some finite steps of computation.
- Approaches for deciding satisfiability:
 - ▶ **Exhaustive Search**: exhaustively search for all possible assignments.
 - ▶ **Deduction**: deduce facts from known facts by iteratively applying proof rules.
 - ▶ **Combination**: Modern SAT solvers are based on **DPLL** that combines search and deduction in an effective way.
- Plan: we will first define the simplistic approach and extend it to DPLL.

Exhaustive Search (Truth Table Method)

- The naive, recursive algorithm for deciding satisfiability.

Algorithm 1 SAT

Input: A PL formula F

Output: Satisfiability (*true*: SAT, *false*: UNSAT)

```
1: if  $F = \top$  then return  $\top$ 
2: else if  $F = \perp$  then return  $\perp$ 
3: else
4:    $P \leftarrow \text{ChooseVar}(F)$ 
5:   return  $\text{SAT}(F\{P \mapsto \top\}) \vee \text{SAT}(F\{P \mapsto \perp\})$ 
```

- When applying $F\{P \mapsto \top\}$ and $F\{P \mapsto \perp\}$, the resulting formulas should be simplified using template equivalences on PL.

$$\begin{array}{ccccccc} \neg \top & \iff & \perp & \neg \perp & \iff & \top & \neg \neg F & \iff & F \\ F \wedge \top & \iff & F & F \wedge \perp & \iff & \perp & F \wedge F & \iff & F \\ F \vee \top & \iff & \top & F \vee \perp & \iff & F & F \vee F & \iff & F \\ & & \dots & & & \dots & & & \dots \end{array}$$

Example 1: Exhaustive Search

Consider the formula $F : (P \rightarrow Q) \wedge P \wedge \neg Q$.

- Choose variable P and

$$F\{P \mapsto \top\} : (\top \rightarrow Q) \wedge \top \wedge \neg Q$$

which simplifies to $F_1 : Q \wedge \neg Q$.

- ▶ $F_1\{Q \mapsto \top\} : \perp$
- ▶ $F_1\{Q \mapsto \perp\} : \perp$

- Recurse on the other branch for P in F .

$$F\{P \mapsto \perp\} : (\perp \rightarrow Q) \wedge \perp \wedge \neg Q$$

which simplifies to $\perp$.

Since all branches end without finding a satisfying assignment, we conclude F is UNSAT.

Example 2: Exhaustive Search

Determine the satisfiability of $F : (P \rightarrow Q) \wedge \neg P$.

Example 2: Exhaustive Search

Determine the satisfiability of $F : (P \rightarrow Q) \wedge \neg P$.

- Choose P and recurse on the first case:

$$F\{P \mapsto \top\} : (\top \rightarrow Q) \wedge \neg \top$$

which is equivalent to $\perp$.

- Try the other case:

$$F\{P \mapsto \perp\} : (\perp \rightarrow Q) \wedge \neg \perp$$

which is equivalent to $\top$.

We conclude F is SAT by the second case. Assigning any value to Q produces a satisfying interpretation:

$$I : \{P \mapsto \text{false}, Q \rightarrow \text{true}\}$$

- SAT solvers convert a given formula F to CNF, and implement various CNF-based optimizations to speed up the solving process.
 - ▶ We will explore several optimizations for CNF formulas.

- SAT solvers convert a given formula F to CNF, and implement various CNF-based optimizations to speed up the solving process.
 - ▶ We will explore several optimizations for CNF formulas.
- Issue: Conversion to an equivalent CNF incurs exponential blow-up in worst-case, resulting in increasing runtime. When?

- SAT solvers convert a given formula F to CNF, and implement various CNF-based optimizations to speed up the solving process.
 - ▶ We will explore several optimizations for CNF formulas.
- Issue: Conversion to an equivalent CNF incurs exponential blow-up in worst-case, resulting in increasing runtime. When?
- Consider converting a formula in DNF into CNF.

$$\begin{aligned} & (F_1 \wedge F_2) \vee (F_3 \wedge F_4) \\ \iff & (F_1 \vee (F_3 \wedge F_4)) \wedge (F_2 \vee (F_3 \wedge F_4)) \\ \iff & (F_1 \vee F_3) \wedge (F_1 \vee F_4) \wedge (F_2 \vee F_3) \wedge (F_2 \vee F_4) \end{aligned}$$

- Another example

$$\begin{aligned} & (F_1 \wedge F_2) \vee (F_3 \wedge F_4) \vee (F_5 \wedge F_6) \\ \iff & (((F_1 \vee F_3) \wedge (F_1 \vee F_4)) \wedge ((F_2 \vee F_3) \wedge (F_2 \vee F_4))) \vee (F_5 \wedge F_6) \\ \iff & (F_1 \vee F_3 \vee F_5) \wedge (F_1 \vee F_4 \vee F_5) \wedge (F_2 \vee F_3 \vee F_5) \wedge (F_2 \vee F_4 \vee F_5) \\ & \wedge (F_1 \vee F_3 \vee F_6) \wedge (F_1 \vee F_4 \vee F_6) \wedge (F_2 \vee F_3 \vee F_6) \wedge (F_2 \vee F_4 \vee F_6) \end{aligned}$$

- n binary disjunctive clauses (n disjunctions of two literals) results in 2^n CNF clauses.

Conversion to an Equisatisfiable Formula in CNF

- Given a formula F , we can convert it into an **equisatisfiable** CNF formula, which increases the size by only a constant factor.
 - ▶ Using the method called Tseitin's transformation.
- F and F' are **equisatisfiable** when F is satisfiable iff F' is satisfiable.
- Key Idea of Tseitin's transformation:
 - (1) introduce new variables to represent the subformulas of F .
 - (2) assert that F 's representative is equivalent to the combination of the subformulas' representatives.

Conversion to an Equisatisfiable Formula in CNF

- Given a formula F , we can convert it into an **equisatisfiable** CNF formula, which increases the size by only a constant factor.
 - Using the method called Tseitin's transformation.
- F and F' are **equisatisfiable** when F is satisfiable iff F' is satisfiable.
- Key Idea of Tseitin's transformation:
 - introduce new variables to represent the subformulas of F .
 - assert that F 's representative is equivalent to the combination of the subformulas' representatives.

(Example) consider the disjunctive formula $G : F_1 \vee F_2$.

$$\begin{aligned} \text{En}(F_1 \vee F_2) &= (P_G \leftrightarrow P_{F_1} \vee P_{F_2}) \\ &= (P_G \rightarrow P_{F_1} \vee P_{F_2}) \wedge (P_{F_1} \vee P_{F_2} \rightarrow P_G) \\ &= (\neg P_G \vee P_{F_1} \vee P_{F_2}) \wedge (\neg(P_{F_1} \vee P_{F_2}) \vee P_G) \\ &= (\neg P_G \vee P_{F_1} \vee P_{F_2}) \wedge (\neg P_{F_1} \wedge \neg P_{F_2}) \vee P_G \\ &= (\neg P_G \vee P_{F_1} \vee P_{F_2}) \wedge (\neg P_{F_1} \vee P_G) \wedge (\neg P_{F_2} \vee P_G) \end{aligned}$$

Conversion to an Equisatisfiable Formula in CNF

Now define the full procedure of Taitin's transformation.

- Let $\mathbf{Rep} : \mathbf{PL} \rightarrow \mathcal{V} \cup \{\top, \perp\}$ be the “representative function”, where $\mathcal{V}$ denotes the set of representative propositional variables.
- $\mathbf{Rep}(F)$ outputs a representative propositional variable P_F such that the truth value of P_F is the same as that of F .

$$\mathbf{Rep}(\top) = \top$$

$$\mathbf{Rep}(\perp) = \perp$$

$$\mathbf{Rep}(P) = P$$

$$\mathbf{Rep}(F) = P_F$$

Conversion to an Equisatisfiable Formula in CNF

En asserts the equivalence between F 's representative and the combination of the subformulas' representatives as a CNF formula.

$$\mathbf{En}(\top) = \top, \quad \mathbf{En}(\perp) = \perp, \quad \mathbf{En}(P) = \top$$

$$\begin{aligned} \mathbf{En}(\neg F) = \\ \text{let } P = \mathbf{Rep}(\neg F) \text{ in} \\ (\neg P \vee \neg \mathbf{Rep}(F)) \wedge (P \vee \mathbf{Rep}(F)) \end{aligned}$$

$$\begin{aligned} \mathbf{En}(F_1 \wedge F_2) = \\ \text{let } P = \mathbf{Rep}(F_1 \wedge F_2) \text{ in} \\ (\neg P \vee \mathbf{Rep}(F_1)) \wedge (\neg P \vee \mathbf{Rep}(F_2)) \wedge (\neg \mathbf{Rep}(F_1) \vee \neg \mathbf{Rep}(F_2) \vee P) \end{aligned}$$

$$\begin{aligned} \mathbf{En}(F_1 \vee F_2) = \\ \text{let } P = \mathbf{Rep}(F_1 \vee F_2) \text{ in} \\ (\neg P \vee \mathbf{Rep}(F_1) \vee \mathbf{Rep}(F_2)) \wedge (\neg \mathbf{Rep}(F_1) \vee P) \wedge (\neg \mathbf{Rep}(F_2) \vee P) \end{aligned}$$

$$\begin{aligned} \mathbf{En}(F_1 \rightarrow F_2) = \\ \text{let } P = \mathbf{Rep}(F_1 \rightarrow F_2) \text{ in} \\ (\neg P \vee \neg \mathbf{Rep}(F_1) \vee \mathbf{Rep}(F_2)) \wedge (\mathbf{Rep}(F_1) \vee P) \wedge (\neg \mathbf{Rep}(F_2) \vee P) \end{aligned}$$

$$\begin{aligned} \mathbf{En}(F_1 \leftrightarrow F_2) = \\ \text{let } P = \mathbf{Rep}(F_1 \leftrightarrow F_2) \text{ in} \\ (\neg P \vee \neg \mathbf{Rep}(F_1) \vee \mathbf{Rep}(F_2)) \wedge (\neg P \vee \mathbf{Rep}(F_1) \vee \neg \mathbf{Rep}(F_2)) \\ \wedge (P \vee \neg \mathbf{Rep}(F_1) \vee \neg \mathbf{Rep}(F_2)) \wedge (P \vee \mathbf{Rep}(F_1) \vee \mathbf{Rep}(F_2)) \end{aligned}$$

Conversion to an Equisatisfiable Formula in CNF

En asserts the equivalence between F 's representative and the combination of the subformulas' representatives as a CNF formula.

$$\text{En}(\top) = \top, \quad \text{En}(\perp) = \perp, \quad \text{En}(P) = P \quad \text{Why } \top?$$

$$\begin{aligned} \text{En}(\neg F) = \\ \text{let } P = \text{Rep}(\neg F) \text{ in} \\ (\neg P \vee \neg \text{Rep}(F)) \wedge (P \vee \text{Rep}(F)) \end{aligned}$$

$$\begin{aligned} \text{En}(F_1 \wedge F_2) = \\ \text{let } P = \text{Rep}(F_1 \wedge F_2) \text{ in} \\ (\neg P \vee \text{Rep}(F_1)) \wedge (\neg P \vee \text{Rep}(F_2)) \wedge (\neg \text{Rep}(F_1) \vee \neg \text{Rep}(F_2) \vee P) \end{aligned}$$

$$\begin{aligned} \text{En}(F_1 \vee F_2) = \\ \text{let } P = \text{Rep}(F_1 \vee F_2) \text{ in} \\ (\neg P \vee \text{Rep}(F_1) \vee \text{Rep}(F_2)) \wedge (\neg \text{Rep}(F_1) \vee P) \wedge (\neg \text{Rep}(F_2) \vee P) \end{aligned}$$

$$\begin{aligned} \text{En}(F_1 \rightarrow F_2) = \\ \text{let } P = \text{Rep}(F_1 \rightarrow F_2) \text{ in} \\ (\neg P \vee \neg \text{Rep}(F_1) \vee \text{Rep}(F_2)) \wedge (\text{Rep}(F_1) \vee P) \wedge (\neg \text{Rep}(F_2) \vee P) \end{aligned}$$

$$\begin{aligned} \text{En}(F_1 \leftrightarrow F_2) = \\ \text{let } P = \text{Rep}(F_1 \leftrightarrow F_2) \text{ in} \\ (\neg P \vee \neg \text{Rep}(F_1) \vee \text{Rep}(F_2)) \wedge (\neg P \vee \text{Rep}(F_1) \vee \neg \text{Rep}(F_2)) \\ \wedge (P \vee \neg \text{Rep}(F_1) \vee \neg \text{Rep}(F_2)) \wedge (P \vee \text{Rep}(F_1) \vee \text{Rep}(F_2)) \end{aligned}$$

Conversion to an Equisatisfiable Formula in CNF

With **Rep** and **En**, we construct the full CNF formula F' that is equisatisfiable to F . Let S_F be the set of all subformulas of F (including F itself).

$$F' : \mathbf{Rep}(F) \wedge \bigwedge_{G \in S_F} \mathbf{En}(G)$$

- Suppose F has size n , where each instance of a logical connective or a propositional variable contributes one unit of size. Then, F' has a size at most $30n + 2$.

The size of F' is linear in the size of F !

- ▶ $|S_F|$ is bound by n .
- ▶ The number of symbols from $\mathbf{En}(F_1 \leftrightarrow F_2)$, which incurs the largest expansion, is 29.
- ▶ Upto one additional conjunction is required per $G \in S_F$.
- ▶ Finally, two extra symbols are required for asserting that $\mathbf{Rep}(F)$ is true.

Example: Conversion to an Equisatisfiable Formula in CNF

$F : (Q_1 \wedge Q_2) \vee (R_1 \wedge R_2)$ is converted into equisat formula

$$F' : P_F \wedge \bigwedge_{G \in S_F} \mathbf{En}(G)$$

where $S_F = \{Q_1, Q_2, Q_1 \wedge Q_2, R_1, R_2, R_1 \wedge R_2, F\}$ and

$$\begin{aligned}\mathbf{En}(Q_1) &= \mathbf{En}(Q_2) = \mathbf{En}(R_1) = \mathbf{En}(R_2) = \top \\ \mathbf{En}(Q_1 \wedge Q_2) &= (\neg P_{(Q_1 \wedge Q_2)} \vee Q_1) \wedge (\neg P_{(Q_1 \wedge Q_2)} \vee Q_2) \\ &\quad \wedge (\neg Q_1 \vee \neg Q_2 \vee P_{(Q_1 \wedge Q_2)}) \\ \mathbf{En}(R_1 \wedge R_2) &= (\neg P_{(R_1 \wedge R_2)} \vee R_1) \wedge (\neg P_{(R_1 \wedge R_2)} \vee R_2) \\ &\quad \wedge (\neg R_1 \vee \neg R_2 \vee P_{(R_1 \wedge R_2)}) \\ \mathbf{En}(F) &= (\neg P_F \vee P_{(Q_1 \wedge Q_2)} \vee P_{(R_1 \wedge R_2)}) \\ &\quad \wedge (\neg P_{(Q_1 \wedge Q_2)} \vee P_F) \wedge (\neg P_{(R_1 \wedge R_2)} \vee P_F)\end{aligned}$$

The Resolution Procedure: Remove Conflicting Literals

- Applicable only to CNF formulas.
- Observation: to satisfy clauses

$$C_1[P] : (l_1 \vee \dots \vee P \vee \dots \vee l_n), \quad C_2[\neg P] : (l'_1 \vee \dots \vee \neg P \vee \dots \vee l'_m)$$

that share the variable P but disagree on its value, either the rest of C_1 or the rest of C_2 must be satisfied. Why?

- The clause $C_1[\perp] \vee C_2[\perp]$ (with simplification) can be added as a conjunction to F to produce an equivalent formula still in CNF.
- The proof rule for **clausal resolution**:

$$\frac{C_1[P] \quad C_2[\neg P]}{C_1[\perp] \vee C_2[\perp]}$$

The new clause $C_1[\perp] \vee C_2[\perp]$ is called the **resolvent**.

- If ever $\perp$ is deduced via resolution, F must be unsatisfiable.
Otherwise, if no further resolutions are possible, F must be satisfiable.

Example 1: Resolution

Consider $F : (\neg P \vee Q) \wedge P \wedge \neg Q$.

- From the resolution

$$\frac{(\neg P \vee Q) \quad P}{Q}$$

we can construct $F' : (\neg P \vee Q) \wedge P \wedge \neg Q \wedge Q$. From the resolution

$$\frac{\neg Q \quad Q}{\perp}$$

we can deduce that F is unsatisfiable.

Example 2: Resolution

Consider $F : (\neg P \vee Q) \wedge \neg Q$.

- The resolution

$$\frac{(\neg P \vee Q) \quad \neg Q}{\neg P}$$

yields $F' : (\neg P \vee Q) \wedge \neg Q \wedge \neg P$.

- Since no further resolutions are possible, F is satisfiable. Indeed, we have a satisfying interpretation $I : \{P \mapsto \text{false}, Q \mapsto \text{false}\}$.

- The Davis-Putnam-Logemann-Loveland algorithm (DPLL) combines the enumerative search and a restricted form of resolution, called **unit resolution**:

$$\frac{l \quad C[\neg l]}{C[\perp]}$$

where l is a literal (i.e., $l = P$ or $l = \neg P$ for some propositional variable P).

- The process of applying this resolution as much as possible is called **Boolean constraint propagation (BCP)**.
- Like the resolution procedure, DPLL operates on PL formulas in CNF.

Example: BCP

Consider $F : P \wedge (\neg P \vee Q) \wedge (R \vee \neg Q \vee S)$ where P is a unit clause.

- Applying the unit resolution

$$\frac{P \quad (\neg P \vee Q)}{Q}$$

yields $F' : Q \wedge (R \vee \neg Q \vee S)$.

- Again, applying the unit resolution

$$\frac{Q \quad (R \vee \neg Q \vee S)}{R \vee S}$$

to F' produces $F'' : R \vee S$, ending the current round of BCP.

DPLL is similar to SAT, except that it begins by applying BCP.

Algorithm 2 DPLL

Input: A PL formula F in CNF

Output: Satisfiability (*true*: SAT, *false*: UNSAT)

- 1: $F \leftarrow \text{BCP}(F)$
 - 2: **if** $F = \top$ **then return** $\top$
 - 3: **else if** $F = \perp$ **then return** $\perp$
 - 4: **else**
 - 5: $P \leftarrow \text{ChooseVar}(F)$
 - 6: **return** $\text{DPLL}(F\{P \mapsto \top\}) \vee \text{DPLL}(F\{P \mapsto \perp\})$
-

Pure Literal Elimination (PLE)

- If variable P appears only positively or only negatively in F , remove all clauses containing an instance of P .
 - ▶ If P appears only positively (i.e., no $\neg P$ in F), replace P by $\top$.
 - ▶ If P appears only negatively (i.e., no P in F), replace P by $\perp$.
- The original formula F and the resulting formula F' are equisatisfiable.
- When only such pure variables remain, the formula must be satisfiable. A full interpretation can be constructed by setting each variable's value based on whether it appears only positively (*true*) or only negatively (*false*).

Algorithm 3 DPLL

Input: A PL formula F in CNF

Output: Satisfiability (*true*: SAT, *false*: UNSAT)

```
1:  $F \leftarrow \text{BCP}(F)$ 
2:  $F \leftarrow \text{PLE}(F)$ 
3: if  $F = \top$  then return  $\top$ 
4: else if  $F = \perp$  then return  $\perp$ 
5: else
6:    $P \leftarrow \text{ChooseVar}(F)$ 
7:   return  $\text{DPLL}(F\{P \mapsto \top\}) \vee \text{DPLL}(F\{P \mapsto \perp\})$ 
```

Example 1: DPLL with BCP and PLE

Consider $F : P \wedge (\neg P \vee Q) \wedge (R \vee \neg Q \vee S)$ in our previous BCP example.

- Recall that applying BCP yields $F'' : R \vee S$, where the unit resolutions correspond to the partial interpretation $\{P \mapsto \text{true}, Q \mapsto \text{true}\}$.
- All variables occur positively, so F is satisfiable:

$$I : \{P \mapsto \text{true}, Q \mapsto \text{true}, R \mapsto \text{true}, S \mapsto \text{true}\}$$

- Branching (lines 6 and 7 in Algorithm 3) is not required in this example.

Example 2: DPLL with BCP and PLE

Consider

$$F : (\neg P \vee Q \vee R) \wedge (\neg Q \vee R) \wedge (\neg Q \vee \neg R) \wedge (P \vee \neg Q \vee \neg R)$$

Example 2: DPLL with BCP and PLE

Consider

$$F : (\neg P \vee Q \vee R) \wedge (\neg Q \vee R) \wedge (\neg Q \vee \neg R) \wedge (P \vee \neg Q \vee \neg R)$$

- No BCP and PLP are applicable.
- Choose Q on the true branch:

$$F\{Q \mapsto \top\} : R \wedge (\neg R) \wedge (P \vee \neg R)$$

We finish this branch, as the unit resolution with R and $\neg R$ deduces $\perp$.

- On the other branch for Q :

$$F\{Q \mapsto \perp\} : (\neg P \vee R)$$

P and R are pure, and thus F is satisfiable with the satisfying interpretation:

$$I : \{P \mapsto \text{false}, Q \mapsto \text{false}, R \mapsto \text{true}\}$$

Summary

- Q. Why computational logic in software engineering?
A. Mathematical basis for systemically analyzing software.
- Syntax and semantics of propositional logic
- Satisfiability and validity
- Equivalence, implications, and equisatisfiability
- Substitution
- Normal forms: NNF, DNF, CNF
- Decision procedures for satisfiability

